Valgrind Fundamentals

Table of Contents

- 0. Summary
- 1. Starting Valgrind
- 2. Basic Options
- 3. Helpful Options
- 4. Valgrind Memcheck Options
- 5. Valgrind Memcheck Output
- 6. References

0. Summary

This document is meant for a quick reference for using **Valgrind** to debug C/C++ programs. It covers essential details for:

- Starting Valgrind
- Basic and helpful options
- Options for Memcheck tool
- Understanding the output of Memcheck

1. Starting Valgrind

The general format to use Valgrind on an executable is the following:

```
valgrind [valgrind-options] program_name [program_args]
```

Valgrind will output the messages in the general format below:

```
==pid== valgrind message
```

2. Basic Options

Options	Description
-h --help	Show help for all options.
--help-debug	Show debugging options.
--version	Show Valgrind version.
-q --quiet	Show only error messages.

3. Helpful Options

Options	Description
--trace-children=<yes\no>	Track children created by parent.
-- track-fds=<yes/no/all/bad>	Track all file descriptor usage.
--log-file=<filename>	Output Valgrind messages to a specified file.

4. Valgrind Memcheck Options

Options	Description
--leak-check=<no\summary\full\yes>	No leak check, summary of leak check, or full detailed leak check
--show-leak-kinds=all	Show every type of leak.

5. Valgrind Memcheck Output

Line	Meaning
at ...	where the error occured.
by ...	call stack leading to the error.
file:line_number	extract source location.
not stack'd, malloc's or free'd	Memory was never valid.
0 bytes inside block size 10	Start of a memory block.

5.1. Invalid Read

The report below shows that we are making an invalid read to a memory that was never valid.

```
==pid== Invalid read of size 10
==pid==   at 0x...: (within file1)
==pid==   by 0x...: (within file2)
==pid==   by 0x...: func1 () (file3:line_number)
==pid== Address 0x... is not stack'd, malloc'd or free'd
```

5.2. Use of Uninitialised Values

The report below shows that we are using a variable that was never initialized inside a conditional statement or check (e.g. if(), while()).

```
==pid== Conditional jump or move depends on uninitialised value(s)
==pid==    at 0x...: func1 () (file1:line_number)
==pid==    by 0x8048472: func2 () (file2:line_number)
```

5.3. Illegal Frees

The report below shows that we are freeing a block of memory twice. First, Valgrind tells you where you made an invalid free call. Second, Valgrind tells you why it was an invalid free "0 bytes inside a block of size 100 free'd" meaning you already free'd that block of memory.

```
==pid== Invalid free()
==pid==    at 0x...: free (vg_clientmalloc.c:577)
==pid==    by 0x...: main (file:10)
==pid==  Address 0x... is 0 bytes inside a block of size 100 free'd
==pid==    at 0x...: free (vg_clientmalloc.c:577)
==pid==    by 0x...: main (file:10)
```

5.4. Overlapping Source and Destination blocks

The report below shows us that the destination address and source address overlap when we try to copy a number of bytes from source to destination.

ex.
dest = 0xbffff294
src = 0xbffff280
n_bytes = 21
Even though dest != src, they still overlap since they are only 14 bytes apart.

```
==pid== Source and destination overlap in memcpy(dest_addr, src_addr,
n_bytes)
==pid==    at 0x...: memcpy (file1:line_number)
==pid==    by 0x...: main (file2:line_number)
```

5.5 Fishy Arguments

The report below shows us that malloc was called with a fishy (weird or suspicious) value. The function malloc takes in an argument of size_t which is unsigned. So if you pass in a negative value it will become extremely large and Valgrind will report it.

```
==pid== Argument 'size' of function malloc has a fishy (possibly negative)
value: -3
==pid==    at 0x...: malloc (file1:line_number)
==pid==    by 0x...: main (file2:line_number)
```

5.6 Memory Leak Detection

Leak Type	Description
Still Reachable	The block of memory is still being pointed to by a pointer. However, the program did not free that block of memory, this is easily fixable by calling free.
Definitely Lost	No pointer at all is pointing to this block of memory, you have completely lost the reference to this block of memory. (e.g. After allocating a block of memory, you set the pointer to NULL, You have lost the reference)
Indirectly Lost	No pointer points at all is pointing to this block of memory. The difference being that the reason it was lost was because some other reference to a block of memory was lost. (e.g. A pointer to the start of the linked list was lost. This indirectly cause the rest of the allocated memory pointed to by the start of the linked list to be lost.)
Possibly Lost	There is a pointer to a block of memory. However, that pointer points to somewhere in the middle of the block of memory and not the start.
Suppressed	Some leaks are suppressed since they are often caused by external libraries and are usually safe to ignore.

6. References

1. Valgrind Documentation